

Operators

Lesson link

https://www.w3schools.com/java/java_operators.asp

Operators allow you to manipulate, compare, and perform calculations using primitive data types, followed closely by **control flow statements**. Understanding these concepts allows you to manipulate data and control the order in which your code executes.

Operators	Explanation																					
Arithmetic Operators These perform standard mathematical calculations.	+ (Addition): Adds values together (e.g., 5 + 3 equals 8). - (Subtraction): Subtracts one value from another (e.g., 10 - 4 equals 6). * (Multiplication): Multiplies values (e.g., 3 * 4 equals 12). / (Division): Divides values (e.g., 15 / 3 equals 5). % (Modulus): Returns the remainder of a division (e.g., 7 % 3 equals 1, because 3 goes into 7 twice with 1 left over).																					
Unary Operators Unary operators are the types that need only one operand to perform any operation like increment, decrement, negation, etc. It consists of various arithmetic, logical and other operators that operate on a single operand. Let's look at the various unary operators in detail and see how they operate.	<table><tr><th>Operators</th><th>Description</th></tr><tr><td>+</td><td>Unary plus operator; indicates positive value (numbers are positive without this, however).</td></tr><tr><td>-</td><td>Unary minus operator; negates an expression.</td></tr><tr><td>++</td><td>Increment operator; increments a value by 1.</td></tr><tr><td>--</td><td>Decrement operator; decrements a value by 1.</td></tr><tr><td>!</td><td>Logical complement operator; inverts the value of a boolean.</td></tr></table>	Operators	Description	+	Unary plus operator; indicates positive value (numbers are positive without this, however).	-	Unary minus operator; negates an expression.	++	Increment operator; increments a value by 1.	--	Decrement operator; decrements a value by 1.	!	Logical complement operator; inverts the value of a boolean.									
Operators	Description																					
+	Unary plus operator; indicates positive value (numbers are positive without this, however).																					
-	Unary minus operator; negates an expression.																					
++	Increment operator; increments a value by 1.																					
--	Decrement operator; decrements a value by 1.																					
!	Logical complement operator; inverts the value of a boolean.																					
Assignment Operators These store values inside variables	<table><tr><th>Operator</th><th>Operation</th><th>Equivalent to</th></tr><tr><td>=</td><td>num = 5</td><td>num = 5</td></tr><tr><td>+=</td><td>num+=5</td><td>num = num+5</td></tr><tr><td>-=</td><td>num-=5</td><td>num = num-5</td></tr><tr><td>*=</td><td>num*=5</td><td>num = num*5</td></tr><tr><td>/=</td><td>num/=5</td><td>num = num/5</td></tr><tr><td>%=</td><td>num%=5</td><td>num = num%5</td></tr></table>	Operator	Operation	Equivalent to	=	num = 5	num = 5	+=	num+=5	num = num+5	-=	num-=5	num = num-5	*=	num*=5	num = num*5	/=	num/=5	num = num/5	%=	num%=5	num = num%5
Operator	Operation	Equivalent to																				
=	num = 5	num = 5																				
+=	num+=5	num = num+5																				
-=	num-=5	num = num-5																				
=	num=5	num = num*5																				
/=	num/=5	num = num/5																				
%=	num%=5	num = num%5																				
Ternary Operator Conditional operator that takes three operands. It's a one-liner replacement for the if-then-else statement	variable = Expression1 ? Expression2 : Expression3																					

Comparison / Relational Operators

They return a boolean result after the comparison and are extensively used in looping statements as well as conditional if-else statements and so on.

Operators	Operations
==	Equal to
!=	Not Equal to
>	Greater than
>=	Greater than equal to
<	Less than
<=	Less than equal to

These compare two values and always result in a boolean (true or false).

- == (Equal to): Checks if two values are the same (e.g., 5 == 5 is true).
- != (Not equal to): Checks if two values are different (e.g., 5 != 3 is true).
- > and < (Greater / Less than): Compares sizes (e.g., 10 > 20 is false).
- >= and <= (Greater / Less than or equal to): Includes the boundary number (e.g., 5 >= 5 is true).

Logical Operators

These combine multiple comparison conditions together to make complex decisions.

One thing to keep in mind is, while using AND operator, the second condition is not evaluated if the first one is false. Whereas while using OR operator, the second condition is not evaluated if the first one is true

Logical Operators in Java

- Logical AND Operator (& and &&)**

Operand 1	Operand 2	Returned Value
False	False	False
False	True	False
True	False	False
True	True	True
- Logical OR Operator (| and ||)**

Operand 1	Operand 2	Returned Value
False	False	False
False	True	True
True	False	True
True	True	True
- Logical NOT Operator (!)**

Operand	Returned Value
False	True
True	False

Operator precedence

Operator precedence determines the order in which different operators are evaluated in a single expression. Think of it as the programming version of **PEMDAS** (Parentheses, Exponents, Multiplication, Division, Addition, Subtraction) from math class.

Without precedence rules, a computer wouldn't know how to solve a mixed expression like $5 + 3 * 2$.

The Core Rules of Precedence

When an expression contains multiple operators, Java evaluates them from highest precedence to lowest precedence.

Here is a simplified table of common Java operators, ranked from **highest priority to lowest**:

Precedence	Operator Type	Symbols	Description
1 (Highest)	Parentheses	<code>()</code>	Forces grouping and overrides everything else.
2	Postfix / Unary	<code>++</code> , <code>--</code> , <code>!</code>	Increments, decrements, and logical NOT.
3	Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>	Multiplication, division, and modulus.
4	Additive	<code>+</code> , <code>-</code>	Addition and subtraction.
5	Relational	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Comparisons.
6	Equality	<code>==</code> , <code>!=</code>	Checking if values match.
7	Logical AND	<code>&&</code>	Evaluates conditions together.
8	Logical OR	<code> </code>	Evaluates optional conditions.
9 (Lowest)	Assignment	<code>=</code> , <code>+=</code> , <code>-=</code>	Storing the final result.

What is Associativity?

When two operators have the **exact same** precedence level, Java uses **associativity** to decide the direction of execution.

- **Left-to-Right:** Most operators (like +, -, *, /) execute from left to right.
- **Right-to-Left:** Assignment operators (like =) execute from right to left.

Arithmetic Mix int result = 10 + 5 * 2;	• Step 1: Multiplication (*) has higher precedence than addition (+). Java calculates 5 * 2 first, which is 10. • Step 2: The expression becomes 10 + 10. • Step 3: Java adds them together to get 20
Using Parentheses to Override int result = (10 + 5) * 2;	• Step 1: Parentheses () have the highest absolute precedence. Java calculates 10 + 5 first, which is 15. • Step 2: The expression becomes 15 * 2. • Step 3: Java multiplies them to get 30.
Mixed Logic and Comparison boolean check = 5 + 3 > 10 && 2 == 2;	• Step 1 (Arithmetic): 5 + 3 is evaluated first. The expression is now 8 > 10 && 2 == 2. • Step 2 (Relational): 8 > 10 is evaluated. It is false. The expression is now false && 2 == 2. • Step 3 (Equality): 2 == 2 is evaluated. It is true. The expression is now false && true. • Step 4 (Logical AND): false && true evaluates to false

Control Statements.

Control statements in Java determine the execution flow of your program. Without them, code simply runs line-by-line from top to bottom.

Java divides control statements into three main categories: **Selection** (Decision-making), **Iteration** (Loops), and **Jump** statements.

1. Selection Statements (Decision Making)

These statements evaluate a boolean condition to decide which block of code to run.

Decision Making statements	Java code
if Statement Executes a block of code only if its condition is true. [	<pre>int age = 20; if (age >= 18) { System.out.println("You can vote."); }</pre>
if-else Statement Provides an alternative block of code if the condition is false.	<pre>int score = 65; if (score >= 70) { System.out.println("Passed"); } else { System.out.println("Failed"); }</pre>
if-else-if Ladder Tests multiple conditions in sequence. It stops at the first true condition.	<pre>int speed = 85; if (speed > 90) { System.out.println("Heavy fine!"); } else if (speed > 70) { System.out.println("Warning ticket."); } else { System.out.println("Safe driving."); }</pre>
switch Statement Compares a single variable against multiple specific constant values (cases). It is cleaner than a long if-else ladder when checking one variable.	<pre>int day = 2; switch (day) { case 1: System.out.println("Monday"); break; case 2: System.out.println("Tuesday"); break; // Matches and prints default: System.out.println("Invalid day"); }</pre>

2. Iteration Statements (Loops)

Loops repeat a block of code as long as a specific condition remains true.

Iteration Statements	Java code
while Loop Checks the condition before executing the loop body. If the condition is false initially, the loop never runs.	<pre>int count = 1; while (count <= 3) { System.out.println(count); count++; // Increments count so the loop eventually ends }</pre>
do-while Loop Executes the loop body first , then checks the condition. This guarantees the code runs at least once .	<pre>int x = 10; do { System.out.println("This runs once even though x > 5"); } while (x < 5);</pre>
for Loop Combines initialization, condition, and increment/decrement into one line. Best used when you know exactly how many times to repeat.	<pre>for (int i = 0; i < 3; i++) { System.out.println("Iteration: " + i); }</pre>
For-Each Loop (Enhanced for) Specifically designed to iterate through all elements in an array or collection without using counter indexes.	<pre>int[] numbers = {10, 20, 30}; for (int num : numbers) { System.out.println(num); }</pre>

3. Jump Statements

Jump statements instantly transfer the program's control to a different part of the code.

Jump Statements	Java code
break Immediately terminates the innermost loop or switch statement.	<pre>for (int i = 1; i <= 5; i++) { if (i == 3) { break; // Stops the loop completely when i is 3 } System.out.println(i); // Prints 1, 2 }</pre>
continue Skips the remaining code inside the loop for the current iteration and jumps directly to the next turn of the loop.	<pre>for (int i = 1; i <= 4; i++) { if (i == 2) { continue; // Skips printing 2 } System.out.println(i); // Prints 1, 3, 4 }</pre>
return Explicitly returns from a method. It stops the execution of the entire method and passes control back to where the method was called.	<pre>public int doubleValue(int number) { return number * 2; // Exits method and sends value back }</pre>

Standard Output and Input

In Java, `System.out` and `System.in` are the primary tools used to communicate with a user through the console or command prompt.

Java has 3 streams called `System.in`, `System.out`, and `System.err` which are commonly used to provide input to, and output from Java applications. They handle data traveling in two opposite directions: **sending information out** to the screen and **bringing information in** from the keyboard.

Study link:

<https://jenkov.com/tutorials/java-io/system-in-out-error.html>

System.out (Standard Output)

`System.out` is used to send data **out** of your program and print it onto the screen.

- **How it works:** `System` is a built-in Java class, and `out` is an object inside it representing your computer screen.
- **The Tools:** It has two main functions you will use constantly.

System.out.println() (Print Line) [1] Prints whatever text you give it, and then moves the cursor to a brand-new line	<pre>System.out.println("Hello"); System.out.println("World"); // Output on screen: // Hello // World</pre>
System.out.print() (Print Only) [1] Prints the text but keeps the cursor on the exact same line . Anything printed next will appear right beside it.	<pre>System.out.print("Hello "); System.out.print("World"); // Output on screen: // Hello World</pre>

System.in (Standard Input)

System.in is used to bring data **into** your program from the outside world (usually by reading what the user types on their keyboard).

- **The Catch:** By itself, System.in is very basic. It only reads typing in raw computer bytes, which is hard for humans to work with.
- **The Fix:** To make it useful, we pass System.in into a helper tool called a **Scanner**. The Scanner takes those raw bytes and translates them into words or numbers we can easily use.

How to use it in code:

Common Scanner Tools for System.in

Once you set up your scanner, you can read different types of user data using these commands:

- scanner.nextInt() — Reads a whole whole number (like 25).
- scanner.nextDouble() — Reads a decimal number (like 19.99).
- scanner.next() — Reads a single word (stops at the first space).
- scanner.nextLine() — Reads an entire sentence or line of text.

```
// 1. Tell Java you want to use the Scanner tool (put this at the very top of your file)
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        // 2. Create the Scanner and hook it up to System.in
        Scanner keyboard = new Scanner(System.in);

        System.out.print("Enter your age: ");

        // 3. Pause the program and wait for the user to type an integer and hit Enter
        int age = keyboard.nextInt();

        System.out.println("Next year you will be: " + (age + 1));

        // 4. Always close the scanner at the end of your program to clean up memory
        keyboard.close();
    }
}
```

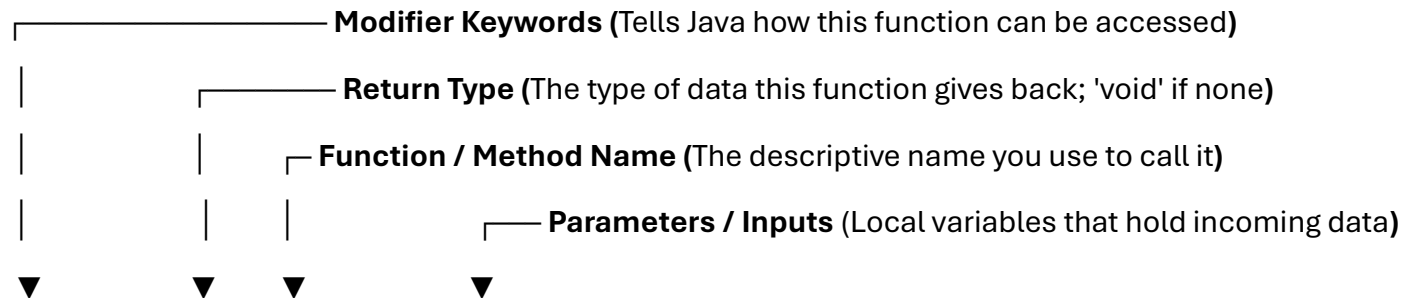
Functions

A **function** (called a **method** in Java) is a reusable block of code that performs a specific task.

Why Use Functions?

- **No Copy-Pasting:** Instead of writing the same 10 lines of code in 5 different places, you write it once inside a function and just call its name.
- **Cleaner Code:** It breaks a massive, messy program down into small, organized, easy-to-read pieces.

The Anatomy of a Basic Function



```
public static int addNumbers (int num1, int num2) {
```

```
    int sum = num1 + num2; ◀— Function Body (The actual code block that does the work)
```

```
    return sum; ◀— Return Statement (output, exit door = data the function will give back to you when it finishes job. Keyword void = If a function doesn't give anything back.)
```

```
}
```

Packages

Packages and **Imports** are Java's system for organizing code file folders and accessing tools.

Think of a large computer. You do not throw every single picture, document, and song onto your desktop. You organize them into folders like /Images or /Music.

- **Packages** are the **folders** used to group related Java files together.
- **Rules for Packages:**
 - **The Declaration:** If a file belongs in a package folder, the very first line of code *must* state that package name.
 - **Naming Convention:** Package names are always completely lowercase. Large companies use their reversed website domain to make them unique (e.g., com.apple.shapes)
- **Example of a Package File:**

Imagine you create a folder named geometry. Inside it, you make a file named Circle.java. The code looks like this:

```
package geometry; // 1. Declares that this file lives in the geometry folder
public class Circle {
    public double calculateArea(double radius) {
        return 3.14 * radius * radius;
    }
}
```

The Built-In Java Packages

Java comes with thousands of pre-made folders filled with tools ready for you to use.

Here are the three most common folders beginner programmers pull tools from: 

Package Folder Name	What It Contains	Example Tools
<code>java.util</code>	Utilities and helpful developer tools.	<code>Scanner</code> (reading keyboards), <code>Random</code> (generating random numbers).
<code>java.time</code>	Tools for tracking dates and clocks.	<code>LocalDate</code> (calendar dates), <code>LocalTime</code> (clock hours).
<code>java.io</code>	Input and Output tools.	<code>File</code> (reading text files saved on your computer hard drive).

imports

If you want to use that Circle tool inside a completely different project folder, your main program will not be able to find it automatically. You must **import** it to bridge the gap.

- import keyword is used to bring pre-written code libraries into your program from massive Java digital libraries.
- By default Java only loads a tiny handful of essential libraries (like basic math and text strings) so your program stays lightweight and fast.
- Place import statements at the very top of your code file, before your class even begins.
- Importing Your Custom Package:

```
import geometry.Circle; // 1. Pulls the Circle tool out of the geometry folder
public class MainProgram {
    public static void main(String[] args) {
```

```
        Circle myShape = new Circle(); // 2. Now you can use it freely!
```

```
    }
}
```

The diagram illustrates the components of the import statement `import java.util.Scanner;` with arrows pointing from descriptive text to the corresponding parts of the code:

- Keyword telling Java to find an outside library** points to `import`.
- The "folder" or package where the tool lives** points to `java.util`.
- The exact "tool" (Class) you want to use** points to `Scanner`.

The full code snippet shown is:

```
import java.util.Scanner;

public class MyProgram {
    // Code goes here...
}
```

Quiz

What is the result of the following expression in Java? int result = 10 + 5 * 2;	20 30 15 25	20 Correct! In Java, multiplication has a higher precedence than addition. So, 5 * 2 is evaluated first to get 10, and then 10 + 10 equals 20.
int x = 5; x += 3;	3 5 8 15	8 Correct! The += is a shorthand assignment operator. x += 3; is exactly the same as writing x = x + 3;. Since x was 5, 5 + 3 gives 8.
Which of the following comparison expressions evaluates to true in Java?	5 > 10 15 == 10 20 <= 20 8 != 8	20 <= 20 Correct! The <= operator means "less than or equal to". Since 20 is equal to 20, the expression evaluates to true.
Assume int a = 10; and int b = 5; . What does the following expression evaluate to? !(a > b) && (b < 20)	True False Null It will cause compilation error	False Correct! First, a > b evaluates to true. Applying the NOT operator! turns it to false. Next, b < 20 is true. Finally, false && true evaluates to false.
What is the result of the following Java expression? boolean b = (5 % 2 == 1) (3 * 2 == 5);	True False This expression throws an Arithmetic exception 0	true Correct! 5 % 2 gives a remainder of 1, so 1 == 1 is true. Because of the short-circuit OR (), if the first part is true, Java knows the whole expression is true without evaluating the second part.
Modulus & Division Mix int result = 15 - 9 % 4 * 2 + 6 / 3; System.out.println(result);	• What prints out? • Hint: Modulus %, multiplication *, and division / all have the same precedence level. Use left-to-right associativity for them before handling addition and subtraction.	
The Logical Operator int x = 5; boolean check = (x++ > 5) && (++x < 10) (x == 6);	• What prints out for check and x? • Hint: Look out for "short-circuit" evaluation. If the first part of an && (AND) statement is false, Java won't even look at the second part! Also, remember the difference between postfix x++ (use then change) and prefix ++x (change then use).	

System.out.println("check = " + check + ", x = " + x);		
String Concatenation System.out.println("Result: " + 5 + 10 * 2);	• What text prints to the console? • Hint: The + operator acts as mathematical addition <i>unless</i> one of the sides is a String, which turns it into text gluing (concatenation).	

Hands on practice - Custom Movie Ticket Calculator.

Instead of doing everything at once, we will build this step-by-step. Each task adds exactly one new coding concept so you can see how it works.

The Goal

Build a program that calculates the price of a movie ticket based on the user's age, checks if they have a student discount coupon, and applies a snack package upgrade.

Task	Code
Task 1: Setting up variables and Math (Operators & Output) Concept: Primitive Data Types, Arithmetic Operators, System.out.println We will start with a baseline fixed price and add a snack upgrade using math. 1. Create a file named TicketCalculator.java. 2. Create 2 variable baseprice and comboPrice. 3. Calculate the total price of ticket	<pre>public class TicketCalculator { public static void main(String[] args) { // Step 1: Create variables for pricing double baseTicketPrice = 12.50; double snackComboPrice = 7.25; // Step 2: Use arithmetic operators to find the total double totalCost = baseTicketPrice + snackComboPrice; // Step 3: Print the results to the screen System.out.println("--- Cinema Receipt ---"); System.out.println("Base Ticket: \$" + baseTicketPrice); System.out.println("Snack Combo: \$" + snackComboPrice); System.out.println("Total Cost: \$" + totalCost); } }</pre>
Task 2: Asking User Questions (System Input) Concept: Using Scanner to read keyboard inputs. Right now, the numbers are hardcoded. Let's make it interactive by asking the user for their name and how many tickets they want. 1. Add import java.util.Scanner; to the very top of your file. 2. Take input - customer name, how many tickets. 3. Calculate total price	<pre>import java.util.Scanner; public class TicketCalculator { public static void main(String[] args) { // Create the Scanner tool to read inputs Scanner input = new Scanner(input); System.out.print("Enter your name: "); String customerName = input.nextLine(); // Reads a line of text System.out.print("How many tickets do you want to buy? "); int ticketCount = input.nextInt(); // Reads a whole integer number</pre>

	<pre> double singleTicketPrice = 12.50; // Multiply ticket count by the price double totalCost = ticketCount * singleTicketPrice; System.out.println("\n--- Receipt for " + customerName + " ---"); System.out.println("Tickets bought: " + ticketCount); System.out.println("Total Balance due: \$" + totalCost); input.close(); } } </pre>
Task 3: Adding Age Discounts (Control Flow / Decision Making) Concept: Using if-else and comparison operators (<, >=) to change prices. Let's make the pricing dynamic. Kids under 12 get a discount, and seniors (65+) get a discount. Everyone else pays full price. Update your code.	<pre> import java.util.Scanner; public class TicketCalculator { public static void main(String[] args) { Scanner input = new Scanner(System.in); System.out.print("Enter your age: "); int age = input.nextInt(); double finalTicketPrice; // Decision logic using comparison operators if (age < 12) { System.out.println("Child discount applied!"); finalTicketPrice = 8.00; } else if (age >= 65) { System.out.println("Senior discount applied!"); finalTicketPrice = 9.00; } else { System.out.println("Regular adult pricing."); finalTicketPrice = 13.50; } System.out.println("Your ticket price is: \$" + finalTicketPrice); input.close(); } } </pre>
Task 4: Organizing with Functions	<pre> import java.util.Scanner; </pre>

Concept: Moving the calculation logic out of main into a reusable function block.

As your code gets bigger, it gets messy. Let's build a dedicated function that calculates the price for us.

```
public class TicketCalculator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your age: ");
        int userAge = input.nextInt();

        // We CALL our custom function here and pass it the
        // user's age
        double price = determinePrice(userAge);

        System.out.println("Final Calculated Price: $" + price);
        input.close();
    }

    // Our new standalone function
    public static double determinePrice(int age) {
        if (age < 12) {
            return 8.00; // Instantly exits the function and passes
            // 8.00 back
        } else if (age >= 65) {
            return 9.00;
        } else {
            return 13.50;
        }
    }
}
```

Task 5: Let It Run Continuously (Loops)

Concept: Using a while loop to let a family calculate multiple different tickets without restarting the program.

Finally, let's wrap everything inside a loop so the app keeps asking for ages until you type a negative number to quit.

```
import java.util.Scanner;

public class TicketCalculator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        boolean keepRunning = true;

        System.out.println("=== Movie Theater Ticket System ===");
        System.out.println("(Type any negative number like -1 to quit)\n");

        while (keepRunning) {
            System.out.print("Enter moviegoer's age: ");
            int age = input.nextInt();
        }
    }
}
```

	<pre> // Jump or break condition if (age < 0) { System.out.println("Exiting system. Have a great movie!"); keepRunning = false; // This breaks the loop condition } else { double ticketPrice = determinePrice(age); System.out.println("Ticket Price: \$" + ticketPrice + "\n"); } } input.close(); } public static double determinePrice(int age) { if (age < 12) return 8.00; if (age >= 65) return 9.00; return 13.50; } } </pre>
Challenge 1. Change the pricing function so that teenagers (ages 13 to 19) get a unique ticket price of \$10.00. 2. Add a simple System.out.println message inside the loop to track how many total tickets have been calculated.	